

# Finding large $k$ -clubs in undirected graphs \*

Maw-Shang Chang, Ling-Ju Hung, Chih-Ren Lin, and Ping-Chen Su

Department of Computer Science and Information Engineering

National Chung Cheng University, Chiayi 62102, Taiwan

{mschang, hunglc, lcj95m, spc98m}@cs.ccu.edu.tw

## Abstract

*Finding cohesive subgroups is an important issue in studying social networks. Many models exist for defining cohesive subgraphs in social networks, such as clique,  $k$ -clique,  $k$ -plex, etc. The concept of  $k$ -club is one of them. A  $k$ -club of a graph is a subset of the vertex set which induces a subgraph of diameter  $k$ . It is a relaxation of a clique, which induces a subgraph of diameter 1. We conducted algorithmic studies on finding a  $k$ -club of size as large as possible. In this paper, we show that for fixed  $k > 1$ , given a parameter  $s$  and a graph  $G$ , to determine whether  $G$  has a  $k$ -club of size greater than or equal to  $s$  is fixed-parameter tractable. We show that one can find a  $k$ -club of maximum size in  $O^*(1.62^n)$  time where  $n$  is the size of the input graph. We implemented a combinatorial branch-and-bound algorithm that finds a  $k$ -club of maximum size and a new heuristic algorithm called IDROP given in this paper. We design a dynamic data structure called  $k$ -DN to speed up the programs. It supports vertex deletion. Given a graph  $G = (V, E)$ , deleting a vertex  $u$  from  $G$ , the data structure maintains all vertices at distance at most  $k$  from  $v$  in  $G[V \setminus \{u\}]$ . From the experimental results that we obtained, we concluded that a  $k$ -club of maximum size can be easily found in sparse graphs and dense graphs. Our heuristic algorithm finds, within reasonable time,  $k$ -clubs of maximum size in sparse and dense graphs in our experiment instances. The gap between the size of a  $k$ -club of maximum size and a  $k$ -club found by IDROP is a constant for the number of vertices that we are able to test.*

## 1 Introduction

All graphs considered in this paper are undirected simple graphs without self loops. Locating cohesive subgroups in social networks helps

people to understand the structures of networks. Unfortunately there is no standard way to define cohesive subgroups in networks. Finding the cohesive subgroup in social networks can be considered as a problem of graph clustering. The graph clustering problem is studied by many researchers [7, 12, 19, 8, 13, 15, 17]. One possible definition of a cohesive subgroup in a graph would be a *clique*, a complete subgraph, but it seems too restricted to consider a clique as a cohesive subgroup in real networks [1, 21]. The distance between a pair of vertices in a graph is the length of a shortest path connecting them. A graph is of *diameter*  $d$  if  $d$  is the maximum distance among all distances between a pair of vertices in the graph. Two models for finding cohesive subgroups in social networks are the *k-clique* [16] and *k-club* [1]. Given a graph  $G$  and an integer  $k$ , a  $k$ -clique of  $G$  is a vertex subset  $K$  such that every pair of vertices in  $K$  have distance not larger than  $k$  in  $G$ . A  $k$ -club in  $G$  is a vertex subset  $S$  such that  $S$  induces a subgraph of diameter  $k$ . Given a graph  $G$  and an integer  $k > 1$ , the maximum  $k$ -club problem is to find a  $k$ -club  $S$  in  $G$  with the maximum cardinality. In this paper, the distance is measured by counting edges, and we will discuss only  $k$ -clubs here. Bourjolly *et al.* showed that the maximum  $k$ -club problem is, unfortunately, NP-hard, even for any fixed  $k > 1$  [4]. They gave some heuristic algorithms for finding  $k$ -clubs [3]. In [4], they gave a branch-and-bound algorithm that finds exactly a maximum  $k$ -club of the input graph.

For a graph  $G = (V, E)$  and  $W \subset V$ , we use  $G[W]$  to denote the subgraph of  $G$  induced by  $W$ . Let  $u, v \in V$  be two vertices in  $G$ . We use  $\text{dist}_G(u, v)$  to denote the distance between  $u$  and  $v$  in  $G$ . For a vertex  $v \notin W$ , we use  $\text{dist}_G(v, W)$  to denote  $\max_{w \in W} \text{dist}_G(v, w)$ . For an integer  $h$ ,  $0 \leq h < n$ , and a vertex  $v$ , the  $h$ -neighborhood of  $v$  in  $G$  the subset of vertices that are at distance no more than  $h$  from  $v$ , *i.e.*,

\*This research is supported by the National Science Council of Taiwan under grant NSC 98-2221-E-194-026-MY3.

$N_h(v) = \{u \mid \text{dist}_G(u, v) \leq h\}$ . A modified  $O$ -notation,  $O^*$ , is used here to bound the running time of those exponential time algorithms asymptotically. For functions  $f$  and  $g$ ,  $f(n) = O^*(g(n))$  if  $f(n) = O(g(n) \cdot \text{poly}(n))$  where  $\text{poly}(n)$  is a polynomial. For more description about the  $O^*$ -notation, we refer to the book [9].

We conducted algorithmic study on the maximum  $k$ -club problem for  $k \geq 2$ . In this paper, we showed that for fixed integer  $k > 1$ , given a positive integer  $s$  as an input parameter and a graph  $G$ , determining whether  $G$  has a  $k$ -club of size greater than or equal to  $s$  is fixed-parameter tractable. We showed that one can find a  $k$ -club of maximum size in  $O^*(1.62^n)$  time where  $n$  is the size of the input graph. We proposed a new heuristic algorithm called IDROP that finds larger  $k$ -clubs than the best known heuristic algorithm called DROP in all graphs. We implemented a combinatorial branch-and-bound algorithm that finds a  $k$ -club of maximum size. To speed up the program, we designed a dynamic data structure that maintains the  $k$ -neighborhood for each vertex of a graph. Given a graph of  $n$  vertices, it can be constructed in  $O(n^2)$  time using  $O(n^2)$  space. It supports both vertex deletion and edge deletion in  $O(km)$  amortized time where  $m$  is the number of edges in the graph. With this data structure, the  $k$ -neighborhood of any vertex can be found in  $O(1)$  time and reported in  $O(\ell)$  time where  $\ell$  is the number of vertices in the  $k$ -neighborhood. From the experimental results that we obtained, we concluded that a  $k$ -club of maximum size can be easily found in sparse graphs and dense graphs. Our heuristic algorithm performs very well. It finds, within reasonable time,  $k$ -clubs of maximum size with very high probability in sparse and dense graphs. On our instances, the gap between the size of a  $k$ -club of maximum size and a  $k$ -club found by the heuristic algorithm is a constant independent of the number of vertices in the graph.

This paper is organized as follows. In Section 2, we give a fixed-parameter algorithm to show that for any fixed  $k > 1$ , the maximum  $k$ -club problem is fixed-parameter tractable. In Section 3, we solve the time complexity problem of the branch-and-bound algorithm in [4, 17]. In Section 4.1, we present a new heuristic algorithm, called IDROP that can be used to find a new lower bound for the branch-and-bound algorithm in [4]. In Section 4.2, we give a reduction rule which can slightly reduce the running time of the branch-and-bound algorithm. In Section 4.3,

we present a new data structure called  $k$ -DN to be used in the implementation of the branch-and-bound algorithm. In Section 5, we give some experiment results to present that our new heuristic algorithm finds larger  $k$ -clubs than existing heuristic algorithms. We also present the performance of our programs implemented based on the branch-and-bound algorithm and with the  $k$ -DN data structure and the data reduction rule.

## 2 The maximum $k$ -club problem is FPT

A problem is *fixed-parameter tractable* (FPT) if given any instance of size  $n$  and a positive integer  $s$ , one can give algorithms to solve it in time  $f(s) \cdot n^{O(1)}$  where  $f(s)$  is a computable function only depending on  $s$ . We call those algorithms *fixed-parameter algorithms*. There are many results about fixed parameter algorithms introduced in [18]. In this section, we give a fixed-parameter algorithm to solve the maximum  $k$ -club problem for any fixed  $k > 1$ .

### The $k$ -club problem

Input: A graph  $G = (V, E)$  and two integers  $k > 1$  and  $s > 1$ .

Output: Whether there is a  $k$ -club  $S$  in  $G$  with  $|S| \geq s$ .

### A fixed-parameter algorithm.

**Step 1:** Let  $\lambda$  be an integer,  $\lambda = s^2$  if  $k$  is even and  $s^3$  if  $k$  is odd. If  $|V| \leq \lambda$ , list  $\binom{|V|}{s}$  subsets of  $V$  and check for each subset  $S \subseteq V$ ,  $|S| = s$ , whether the diameter of  $G[S]$  is at most  $k$  by computing all pairs shortest paths.

**Step 2:** If there exists a vertex  $v$  in  $G$ ,  $|N_{\lfloor k/2 \rfloor}(v)| \geq s$ , then we simply take  $S = N_{\lfloor k/2 \rfloor}(v)$ .

**Step 3:** For every  $v \in V$ , check  $\binom{|N_k(v)|}{s}$  subsets of  $N_k(v)$  to see whether there exists a  $k$ -club in  $N_k(v)$ .

**Lemma 1.** Suppose for each  $v$  in  $G$ ,  $|N_{\lfloor k/2 \rfloor}(v)| < s$ . If  $k$  is even,  $|N_k(v)| < s^2$ . If  $k$  is odd,  $|N_k(v)| < s^3$ .

**Proof.** Suppose that  $k$  is even, for each  $v$  in  $G$ ,  $|N_{k/2}(v)| < s$ . For each  $w \in N_{k/2}(v)$ ,  $|N_{k/2}(w)| < s$ , hence

$$|N_k(v)| \leq \sum_{w \in N_{k/2}(v)} (s-1) \leq (s-1)^2 < s^2.$$

This shows that when  $k$  is even, for each  $v$  in  $G$ ,  $|N_k(v)| < s^2$ .

Suppose that  $k$  is odd, for each  $v$  in  $G$ ,  $|N_{(k-1)/2}(v)| < s$ . For each  $w \in N_{(k-1)/2}(v)$ ,  $|N_{(k-1)/2}(w)| < s$ ,

$$|N_{k-1}(v)| \leq \sum_{w \in N_{(k-1)/2}(v)} (s-1) \leq (s-1)^2 < s^2.$$

Moreover, every  $w \in N_{k-1}(v)$  has at most  $s-1$  neighbors,

$$|N_k(v)| \leq \sum_{w \in N_{k-1}(v)} s-1 \leq s^2(s-1) < s^3.$$

This shows that when  $k$  is odd, for each  $v$  in  $G$ ,  $|N_k(v)| < s^3$ .  $\square$

**Theorem 1.** *Finding a maximum  $k$ -club in an undirected graph is fixed-parameter tractable for any fixed  $k > 1$ .*

**Proof.** The worst case happens when  $k$  is odd. So we only need to show the complexity of the algorithm when  $k$  is odd. By Lemma 1,  $|N_k(v)| < s^3$  when  $k$  is odd. It takes  $O(s^3)$  time to check whether an induced subgraph with  $s$  vertices is a  $k$ -club. For each  $v$ , the algorithm checks at most  $\binom{s^3}{s}$  subsets of  $N_k(v)$  to see if there exists a  $k$ -club of size  $s$ , and  $O(\binom{s^3}{s} \cdot s^3)$  time is needed for finding a  $k$ -club in  $N_k(v)$ . Hence the running time of the algorithm is  $O(n \cdot s^{3s+3})$ . This shows that finding a maximum  $k$ -club in an undirected graph is fixed-parameter tractable for any fixed  $k > 1$ .  $\square$

### 3 An $O^*(1.62^n)$ exact algorithm

Bourjolly *et al.* [4] gave a branch-and-bound algorithm for solving the maximum  $k$ -club problem without analyzing the time complexity. In this section, we analyze the worst-case time complexity of this algorithm. At the root node of the search tree, two branches are generated corresponding to removing or keeping the vertex selected. The algorithm selects a vertex  $v$  with smallest size of  $k$ -neighborhood for branching. If the input graph is not a  $k$ -club, then there is at least a vertex not in the  $k$ -neighborhood of  $v$ . Notice that  $|N_k(v)| < |V|$  and we always consider  $v$  in which  $N_k(v)$  contains least number of vertices in the graph. In each node of the search tree, there are two branches corresponding to removing or keeping  $v$ . In the branch of keeping  $v$ ,

all vertices not in the  $k$ -neighborhood of  $v$  must be removed from the input graph since they will not be a member of any solution obtained in this branch. Let  $S, T, R$  be the three sets of vertices representing the solution set, non-solution set, and undecided set. Initially,  $S = T = \emptyset$ ,  $R = V$ , and for every branch node with subgraph  $G[W]$ , there are two branches for choosing a pair of vertices  $u, v \in R$  which  $\text{dist}_{G[W]}(u, v) > k$ .

1.  $T = T \cup \{v\}$ ,  $R = R \setminus \{v\}$
2.  $S = S \cup \{v\}$ ,  $T = T \cup \{u\}$ ,  $R = R \setminus \{v, u\}$

Note that the depth of the search tree is related with  $|R|$ . Once  $R = \emptyset$ , we see that we reach leaves, and the current induced subgraph  $G[S]$  must be a  $k$ -club. The maximum  $k$ -club can be found when all leaves of the search tree are visited.

Let  $T(n)$  denote the time complexity of the exact algorithm where  $n$  is the number of vertices in  $G$ . Then,

$$T(n) = T(n-1) + T(n-2) + O(n^3),$$

where  $O(n^3)$  is the time spent to compute all-pairs distance and to select a vertex  $v$  considered in this branch. The term  $T(n-2)$  measures the time for the branch that keeps  $v$  and removes all vertices not in the  $k$ -neighborhood of  $v$  from the graph. The term  $T(n-1)$  measures the time for the branch that removes  $v$  from the graph. By solving the recurrence equation, we see that the time complexity of this algorithm is  $O^*(1.62^n)$ .

**Theorem 2.** *A maximum  $k$ -club can be found in  $O^*(1.62^n)$  time, where  $n$  is the number of vertices in the input graph.*

### 4 Algorithmic tricks

To implement the branch-and-bound algorithm to run fast, we incorporate four algorithmic tricks. One is to design a new heuristic algorithm that can find larger  $k$ -clubs than existing heuristic algorithms. We use the  $k$ -club found by this new heuristic algorithm as an initial feasible  $k$ -club. The second is to design simple and efficient reduction rules used in each node of the search tree to reduce the size of the subproblem before branching. The third is to have an efficient algorithm that computes an upper bound of sizes of  $k$ -clubs in the graph associated with a node of the search tree. If this bound is not better than the

best solution found so far, then the branch-and-bound algorithm will terminate this node. In [4] the authors find the  $k$ -clique number of the graph associated with each node of the search tree as an upper bound. The main drawback of this approach is that the maximum  $k$ -clique problem, which is NP-hard, must be solved to optimality to obtain a valid upper bound. Inspired by branch-and-bound algorithms for solving the maximum clique problem, we came out with the idea using the  $k$ -coloring number as an upper bound. Later we found the same technique was independently used in [17]. The  $k$ -coloring number of a graph is the minimum number of colors one can use to color vertices of the graph such that two vertices that are at a distance no more than  $k$  are in different colors. The size of any feasible  $k$ -coloring of a graph can be used as an upper bound of the size of  $k$ -clubs in the graph. During the execution, a branch-and-bound algorithm keeps on modifying the input graph and storing partial results. Hence the fourth trick is to design efficient dynamic data structures for storing the input graph and maintaining properties of the graph. In each node of the branch-and-bound algorithm for finding a maximum  $k$ -club, it tests whether the input graph is already a  $k$ -club and selects a vertex with minimum  $k$ -neighborhood for branching if the input graph is not a  $k$ -club. Hence computing the  $k$ -neighborhood of each vertex is a major procedure of the algorithm. Therefore we design a dynamic data structure that stores the  $k$ -neighborhood of every vertex supporting vertex deletion. We believe that this data structure is not only useful in the implementation of the branch-and-bound algorithm, it also has its own interest. In the rest of this section, we will describe our new heuristic algorithm, reduction rules and the dynamic data structure.

#### 4.1 A new heuristic algorithm

In [3], Bourjolly *et al.* gave three heuristic algorithms for finding a  $k$ -club in an undirected graph called CONSTELLATION, DROP, and  $k$ -CLIQUE & DROP. For readers' convenience, we briefly describe them. Algorithm DROP works as follows: While the graph is not a  $k$ -club, repeatedly remove a vertex with smallest size of  $k$ -neighborhood. Algorithm CONSTELLATION is based on the following definition. Recursively define the weight  $W(t)$ ,  $2 \leq t \leq k$ , for a graph  $G$  as follows: If  $t = 2$ , let  $W(t) = N_G[v]$ , where  $v$  is a vertex of maximum degree in  $G$ . For  $2 < t \leq k$ ,

let  $W(t) = W(t-1) \cup N_G(w)$ , where  $w$  is a vertex in  $W(t-1)$  adjacent to maximum number of vertices not in  $W(t-1)$ . Algorithm CONSTELLATION computes  $W(t)$  iteratively and outputs  $W(k)$ . It is not hard to verify that  $W(k)$  is indeed a  $k$ -club of  $G$ . The third algorithm is called  $k$ -CLIQUE & DROP. It first determines a maximum  $k$ -clique  $K$  in  $G$  and then call DROP to find a  $k$ -club in  $G[K]$ .

Bourjolly *et al.* showed that the running times of DROP and CONSTELLATION are  $O(n^3m)$  and  $O(k(n+m))$ , respectively. Since the problem to find a maximum  $k$ -clique is NP-hard, of course the running time of algorithm  $k$ -CLIQUE & DROP is not polynomial unless  $P=NP$ . According to the experiments done by the authors, CONSTELLATION finds better solution in sparse graphs and DROP finds better solution in dense graphs. And the solutions found by the third algorithm are dominated by one or the other of the first two algorithms in most cases. Even in the case that the third algorithm finds a better solution than the first two, it only finds a slightly better solution. The slightly better solution does not justify taking exponential time.

Now we describe our new heuristic algorithm called Iterative DROP (IDROP for short). This algorithm is a modification of the third algorithm: For each vertex  $v$  in the graph we call DROP to compute a  $k$ -club in the subgraph induced by the  $k$ -neighborhood of  $v$  and then output a best  $k$ -club among them. The running time of this algorithm is polynomial. The idea behind the algorithm is that a maximum  $k$ -club may includes some vertices whose  $k$ -neighborhood is not large but they are excluded from the solution obtained by DROP. Fig. 1 shows an example graph in which DROP excludes those pendant vertices in finding a 2-club and therefore it returns a 2-club of size smaller than that the one found by IDROP. On our instances, IDROP always finds solutions that are at least as good as DROP. From experimental results we observe that this algorithm never finds a solution much inferior to those found by CONSTELLATION in our test instances.

#### 4.2 A reduction rule

Let  $C$  be a  $k$ -club of graph  $G$  and  $v \in C$ . Then  $C$  is a subset of the  $k$ -neighborhood of  $v$ . During the execution, a branch-and-bound algorithm maintains a best solution found so far. Suppose the size of this solution is  $\max$ . If the size of the

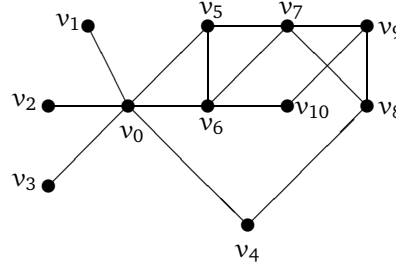


Figure 1: In this example graph, DROP finds 2-club  $\{v_5, v_6, v_7, v_8, v_9, v_{10}\}$  but IDROP finds 2-club  $\{v_0, v_1, v_2, v_3, v_4, v_5, v_6\}$  of bigger size.

$k$ -neighborhood of a vertex in the graph associated with a node of the search tree is no more than  $\max$ , then we can remove the vertex from the graph. We found this reduction rule is very useful in case the variance of vertex degrees of the graph are large.

### 4.3 A dynamic $k$ -neighborhood data structure

In the branch-and-bound algorithm for solving the maximum  $k$ -club problem described in Section 3, for each branch node of the branch-and-bound search tree we need to compute the distance between all-pairs vertices in the current given graph. This would introduce redundant computations because in any node of the branch-and-bound search tree except the root the graph considered by now only has slight differences compared with the graph associated with its parent node. A dynamic all-pairs shortest path algorithm is needed for reducing the entire computation time. Some dynamic data structures were introduced to maintain all-pairs shortest paths [5, 20]. We observe the branch-and-bound algorithm and then discover that it is not necessary to maintain the all-pairs shortest paths, because in the entire procedure of solving the maximum  $k$ -club problem we do not have to keep the information that a pair of vertices have distance more than  $k$ . So to speed up the performance of the program, we need a dynamic data structure that supports vertex deletion and for each vertex in the graph it maintains all  $k$ -neighborhoods. In this paper we independently designed a data structure called  $k$ -DN for our special purpose. Later an anonymous reviewer reminded us that the dynamic data structure described in [14] for directed graphs that supports edge deletion only and maintains all-pairs shortest paths of length

bounded by  $k$  can be applied for our purpose.

Since we solve the problem on undirected graphs, for the sake of completeness we describe the data structure  $k$ -DN in this section. The data structure  $k$ -DN for undirected graphs is able to maintain the  $k$ -neighborhood of each vertex after a series of vertex deletion. In  $k$ -DN, each vertex of the input graph is identified by a number in  $\{1, 2, \dots, n\}$  where  $n$  is the number of vertices of the input graph. The basic building block of  $k$ -DN is an array of  $n$  list nodes for each vertex  $i$ . Based upon this array we construct a double linked list storing all vertices that are at distance  $d$  from vertex  $i$  for each  $1 \leq d \leq k$ . Any vertex  $u$  other than  $i$  will be in at most a list of  $i$  and no matter  $u$  is in any list of  $i$ , the list node storing  $u$  will be located at the  $u$ -th position of the array. For each vertex  $u$  in a list of vertex  $i$ , we also store the distance between  $u$  and  $i$  in the graph and other data that are important to our algorithm. For each  $u$  at distance  $d$ ,  $1 \leq d \leq k$ , from vertex  $i$ , we also maintain the number of vertices that are at distance  $d - 1$  from vertex  $i$  and are adjacent to vertex  $u$ . We call this number the upper degree of  $u$  with respect to vertex  $i$ . In Fig. 2, we give an example to present a basic block of the  $k$ -DN data structure with respect to vertex  $v_0$  in the graph shown in Fig. 1.

By using double linked list and a fixed-location for each node, a vertex can be inserted into or deleted from a list in constant time. We can locate the set of vertices that are at distance  $d$  from vertex  $i$  in constant time and report them in  $O(\ell)$  time where  $\ell$  is the number of vertices that are at distance  $d$  from vertex  $i$ . In constant time, we can report whether the distance between two vertices is greater than  $k$  or obtain the distance between them if it is no more than  $k$ . The space used by the data structure is  $O(n^2)$ . It can be constructed in  $O(nm)$  time as follows: First allocate

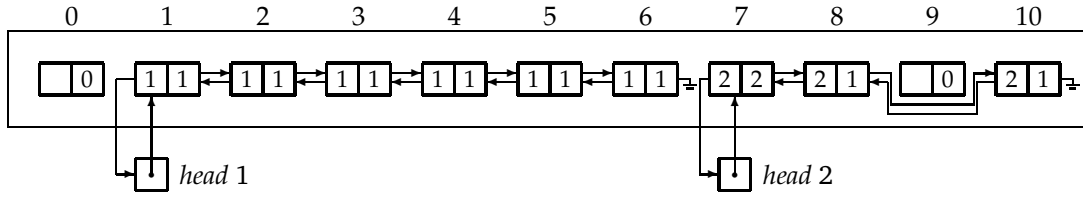


Figure 2: This presents a basic building block of the  $k$ -DN data structure for  $k = 2$ . The corresponding vertex of this example is vertex  $v_0$  in the graph shown in Fig. 1. The double linked list starting from *head 1* stores those vertices adjacent to  $v_0$  and the double linked list starting from *head 2* stores those vertices at distance two from  $v_0$ . For  $j \in \{0, 1, 2, \dots, 10\}$ , the left entry of  $j$ -th position of the array stores the distance between  $v_j$  and  $v_0$  and the right entry stores the upper degree of  $v_j$  with respect to  $v_0$ , for example, the right entry of position 7 is 2 since  $v_7$  has two neighbors  $v_5$  and  $v_6$  in the list starting from *head 1*.

an array of list nodes for each vertex and initialize them. Then run a breadth-first traversal for each vertex  $i$  to obtain the distance between  $i$  and any other vertex and then insert every vertex into the list storing all vertices at distance  $d$  from vertex  $i$  if it is at distance  $d$  from  $i$ . This step takes  $O(n + m)$  time per vertex, where  $m$  is the number of edges of the graph. If we store the number of vertices in the list for each list and the number of vertices in the  $k$ -neighborhood of every vertex  $i$ , then we can verify whether the input graph is a  $k$ -club by checking if the  $k$ -neighborhood of every vertex equals the vertex set of the graph.

The critical part is how to update efficiently the  $k$ -neighborhood of every vertex after deleting a vertex  $i$  from the graph. Clearly, only the  $k$ -neighborhood of vertices in the  $k$ -neighborhood of vertex  $i$  affected by the deletion. In the following we show how to update the  $k$ -neighborhood of vertex  $j$  when vertex  $i$  is deleted. Let  $d$  be the distance between  $i$  and  $j$ . Of course  $d \leq k$ .

Remove vertex  $i$  from the  $k$ -neighborhood of vertex  $j$  first. If  $d = k$  then the procedure terminates. In the following assume  $d < k$ . The procedure maintains two sets  $U$  and  $U'$  and a variable  $h$ . Initially let  $h = d + 1$ ,  $U' = \emptyset$  and  $U$  be the set of vertices that are at distance  $d + 1$  from vertex  $j$  and have no neighbors at distance  $d$  from vertex  $j$  after the deletion. They are neighbors of vertex  $i$  and the upper degrees of them with respect to vertex  $j$  before the deletion is 1. Hence the distance before the deletion between vertex  $j$  and any vertex in  $U$  is  $h$  and will be increased by the deletion. After the initialization, the procedure repeats the following steps to update the list of vertex  $j$  one by one while  $h \leq k$  and  $U$  or  $U'$  is not the empty set:

Step 1. Remove the set of vertices in  $U$  from the list of vertex  $j$  storing the set of vertices that are at distance  $h$  from vertex  $j$ .

Step 2. The distance between vertex  $j$  and any vertex  $s \in U'$  after the deletion is yet to be determined. For each  $s \in U'$  compute the number of vertices that are at distance  $h - 1$  from vertex  $j$  after the deletion and are adjacent to vertex  $s$ , insert  $s$  into the list of vertex  $j$  storing the set of vertices that are at distance  $h$  from vertex  $j$  and remove  $s$  from  $U'$  if the number is greater than 0.

Step 3. Let  $S = \emptyset$ .

Step 4. If  $h < k$  compute  $S$  as follows:  
For each vertex  $s \in U$  and each neighbor  $t$  of  $s$  at distance  $h + 1$  from vertex  $j$  before the deletion, decrease the upper degree of  $t$  with respect to  $j$  by one and insert  $t$  into  $S$  if the upper degree of  $t$  with respect to vertex  $j$  becomes 0. Therefore  $S$  is the set of vertices that are at distance  $h + 1$  from vertex  $j$  before the deletion such that the distance between vertex  $j$  and any vertex in  $S$  is increased after the deletion.

Step 5. Let  $U' = U' \cup U$ .

Step 6. Let  $U = S$  and  $h = h + 1$ .

The correctness of the above procedure can be proved by induction. We claim that (1) before each iteration, the lists storing vertices that are at distance less than  $h$  from vertex  $j$  after the deletion are valid; (2) the distance after the deletion between vertex  $j$  and a vertex  $s$  at distance  $h$  from vertex  $j$  before the deletion will be increased by

the deletion only if  $s \in U$ ; and (3) the distance between vertex  $j$  and any vertex  $s \in U'$  is at least  $h$ . It is easy to check that all claims are true before each iteration. Hence the procedure correctly updates the  $k$ -neighborhood of every vertex.

Next we analyze the time complexity of the update procedure. It is easy to see that it takes  $O(\deg(i))$  to delete vertex  $i$  from the lists of vertex  $j$  that is in the  $k$ -neighborhood of  $i$ , where  $\deg(i)$  denote the degree of vertex  $i$ . For each vertex  $s$  in the  $k$ -neighborhood of vertex  $j$  it takes  $O(k \cdot \deg(i))$  time in total for all vertex deletions since it takes  $O(f \cdot \deg(i))$  time in Step 2 if the distance between  $s$  and vertex  $j$  is increased by  $f$  in a deletion. And the total distance increased is at most  $k - 1$ . Thus the overall running time of the procedure of all deletions in updating the  $k$ -neighborhood of a vertex is  $\sum_i k \cdot \deg(i)$  and hence the overall running time of the update procedure for all deletions is  $\sum_i \sum_i k \cdot \deg(i) = O(knm)$ . Thus we have the following result:

**Theorem 3.** *The dynamic  $k$ -neighborhood of all vertices of a graph of  $n$  vertices and  $m$  edges can be updated in  $O(km)$  amortized time for each vertex deletion.*

## 5 Experiment Results

We have implemented the branch-and-bound algorithm described in Section 3 including all tricks given in Section 4. The branch-and-bound algorithm takes the solution found by IDROP as an initial lower bound and the greedy coloring as an upper bound. The program is implemented in C and all experiments were conducted on an ASUS<sup>®</sup> workstation AS-D900 with Intel<sup>®</sup> Core i7 2.67 GHz and 12.00 GB ram and performed with single threaded workloads. The execution time of those experiment results is measured in seconds.

In this paper, we use a set of random graphs, DIMACS benchmark graphs [6], and Erdős collaboration networks [2, 11] as our test instances. We generate these random graphs by using the algorithm of Gendreau *et al.* [10], the random graph generator is controlled by two density parameters  $a$  and  $b$  ( $0 \leq a \leq b \leq 1$ ). The random graphs are examined to be connected graphs, in case the tests are not accurate. We generate graphs having density between 0.05 to 0.20 with gap 0.05. For each kind of density, we define three pairs of density parameters to generate 100 random graphs, each

of them has 150 vertices. We use those random graphs to test the performance of our programs for finding  $k$ -clubs,  $k = 2, 3, 4$ . Since those random graphs of density greater than 0.01 are almost a 3-club and a 4-club, on finding 3-clubs and 4-clubs we test our programs on random graphs with small density. For 3-clubs, we only take random graphs with density 0.05 and density 0.10 as our test instances; for 4-clubs, we only take random graphs of density 0.05 as our test instances.

DIMACS graphs are the benchmark graphs for the maximum clique problem. Since the maximum  $k$ -club problem is a clique relaxation related problem, we can also use these DIMACS graphs to test the efficiency of our program.

Erdős collaboration networks are obtained from the collaboration of Erdős and his coauthors and also those coauthors of Erdős' coauthors. In this network an author is a vertex of graphs. Two authors are connected with an edge if they are coauthors in some paper. Authors who are coauthors of Erdős are called Erdős 1 vertices and they are adjacent to Erdős. Authors who are not coauthors of Erdős but are coauthors of some of Erdős' coauthors are called Erdős 2 vertices. In the experiment, we use ERDOS- $x$ - $y$  to denote which network we consider where  $x$  represents the last two digits of the year that the network was constructed and  $y$  represents the largest Erdős number of an author. For example, the vertices in ERDOS-99-2 correspond to 6100 authors who are either coauthors of Erdős or coauthors of one of Erdős' coauthors. Note that we use the induced subgraphs where the vertex corresponding to Erdős is excluded.

In Table 1, we compare the average size of  $k$ -clubs found by three heuristic algorithms, CONSTELLATION, DROP, and IDROP in 100 random graphs. We also list the average size of maximum  $k$ -clubs of those random graphs, for  $k = 2, 3, 4$ . From these results, we see that in most cases IDROP returns large  $k$ -clubs which are very close to the optimum solution. We developed two programs based on the same branch-and-bound algorithm described in Section 3. One is using the size of the solution found by DROP as the initial lower bound and the other is using the size of the solution found by IDROP. In Table 2, we compare the performance of them. Since the solution of IDROP is at least as good as the solution found by DROP, the number of leaves in the search tree with the initial lower bound found by IDROP is always less than or equal to the number of leaves in the search tree with the initial

lower bound found by DROP. We also implement the other two programs based on the branch-and-bound algorithm in Section 3, one is applying the reduction rule given in Section 4.2, but the other is not. Both of them use the k-DN data structure. In Table 3, we observe that by using this reduction rule the number of leaves in the branch-and-bound search tree can be decreased and for those random graphs with density 0.15 and 0.20, the running time can be slightly improved. In Table 4, we test our programs on 100 random graphs of 150 vertices to compare the performance of the program with k-DN data structure and the program without k-DN data structure. In these experiment results, we see that the program with k-DN data structure performs better than the other one. In Table 5 and 6, we list the size of k-clubs found by the branch-and-bound algorithm, DROP, and IDROP in some DIMACS graphs and Erdős collaboration networks. In most cases of these benchmark graphs, the solution found by IDROP is exact the size of maximum k-clubs.

## References

- [1] R. Alba. A graph-theoretic definition of a sociometric clique. *Journal of Mathematical Sociology* **3** (1973), pp. 113–126.
- [2] V. Batagelj, Network/Pajek Graph Files. <http://vlado.fmf.uni-lj.si/pub/networks/pajek/data/gphs.htm>
- [3] J.-M. Bourjolly, G. Laporte, and G. Pesant. Heuristics for finding k-clubs in an undirected graph. *Computer & Operation Research*, **27** (2000), pp. 559–569.
- [4] J.-M. Bourjolly, G. Laporte, and G. Pesant. An exact algorithm for maximum k-club problem in an undirected graph. *European Journal of Operational Research*, **138** (2002), pp. 21–28.
- [5] C. Demetrescu, A new approach to dynamic all pairs shortest paths, *Journal of the ACM*, **51** (2004), pp. 968–992.
- [6] DIMACS: Maximum clique, graph coloring, and satisfiability, Second DIMACS implementation challenge (1995), <http://dimacs.rutgers.edu/Challenges/>.
- [7] B. Everitt. *Cluster Analysis*. New York: Halsted Press, 1980.
- [8] G. W. Flake, R. E. Tarjan, and K. T. Sioutouliklis, Graph clustering and minimum cut trees. *Internet Mathematics*, **1** (2004), pp. 385–408.
- [9] F. Fomin and D. Kratsch, *Exact Exponential Algorithms*, Springer, 2010.
- [10] M. Gendreau, P. Soriano, and L. Salvail, Solving the maximum clique problem using a tabu search approach, *Annals of Operations Research*, **41** (1993) pp. 385–403.
- [11] J. Grossman, P. Ion, and R. D. Castro, The Erdős Number Project. <http://www.oakland.edu/enp>
- [12] A. K. Jain and R. C. Dubes. *Algorithms for clustering Data*. Englewood Cliffs, NJ: Prentice-Hall, 1988.
- [13] R. Kannan, S. Vempala, and A. Vetta. On Clusterings-Good, Bad and Spectral. *Journal of the ACM*, **51** (2004), pp. 497–515.
- [14] V. King, Fully dynamic algorithms for maintaining all-pairs shortest paths and transitive closure in digraph, *Proceedings of FOCS 1999*, pp. 81–91.
- [15] S. T. Kuan, B. Y. Wu, and W. J. Lee, Finding friend groups in Blogosphere, *Proceedings of 22nd International Conference on Advanced Information Networking and Application*, pp. 1046–1050, 2008.
- [16] R. Luce. Connectivity and generalized cliques in sociometric group structure. *Psychometrika*, **15** (1950), pp. 169–190.
- [17] F. Mahdavi and B. Balasundaram, On inclusionwise maximal and maximum cardinality k-clubs in graphs. <http://iem.okstate.edu/baski/files/DISCO-k-clubs-2010-02-11.pdf> (2010)
- [18] R. Niedermeier, *Invitation to Fixed-Parameter Algorithms*, Oxford University Press, 2006.
- [19] J. Scott, *Social Network Analysis- A Handbook*, Sage Publication, London, 2000.
- [20] Valerie King. Fully dynamic algorithms for maintaining all-pairs shortest paths and transitive closure in digraphs. *Proceedings of the 40th Annual Symposium on Foundations of Computer Science* (1999), pp. 81–89.
- [21] S. Wasserman and K. Faust. *Social Network Analysis*. Cambridge University Press, 1994.

Table 1: This table shows the average size of  $k$ -clubs,  $k = 2, 3, 4$ , found by CONSTELLATION, DROP, and IDROP in a set of 100 random graphs of  $|V| = 150$ . We use bold texts to mark those sizes of largest  $k$ -clubs found amongst the three heuristics.

| k | Density | [a, b]         | CONSTELLATION |      | DROP          |      | IDROP         |      | Exact  |
|---|---------|----------------|---------------|------|---------------|------|---------------|------|--------|
|   |         |                | Size          | Time | Size          | Time | Size          | Time |        |
| 2 | 0.05    | [0.000, 0.100] | <b>16.35</b>  | 0.00 | 8.10          | 0.00 | <b>16.35</b>  | 0.08 | 16.35  |
|   |         | [0.025, 0.075] | <b>16.31</b>  | 0.00 | 8.22          | 0.00 | <b>16.31</b>  | 0.08 | 16.31  |
|   |         | [0.050, 0.050] | <b>16.36</b>  | 0.00 | 8.53          | 0.00 | <b>16.36</b>  | 0.08 | 16.36  |
|   | 0.10    | [0.050, 0.150] | <b>26.47</b>  | 0.00 | 14.34         | 0.00 | 21.10         | 0.15 | 26.47  |
|   |         | [0.075, 0.125] | <b>26.35</b>  | 0.00 | 14.02         | 0.00 | 21.09         | 0.15 | 26.35  |
|   |         | [0.100, 0.100] | <b>26.15</b>  | 0.00 | 13.94         | 0.00 | 20.69         | 0.15 | 26.15  |
|   | 0.15    | [0.100, 0.200] | 35.50         | 0.00 | 34.17         | 0.00 | <b>44.13</b>  | 0.23 | 52.73  |
|   |         | [0.125, 0.175] | 35.17         | 0.00 | 32.73         | 0.00 | <b>42.98</b>  | 0.24 | 52.91  |
|   |         | [0.150, 0.150] | 35.65         | 0.00 | 32.98         | 0.00 | <b>43.58</b>  | 0.22 | 51.85  |
|   | 0.20    | [0.100, 0.300] | 44.24         | 0.00 | 130.71        | 0.00 | <b>133.85</b> | 0.06 | 134.31 |
|   |         | [0.150, 0.250] | 44.14         | 0.00 | 131.03        | 0.00 | <b>134.52</b> | 0.06 | 134.89 |
|   |         | [0.200, 0.200] | 44.77         | 0.00 | 131.83        | 0.00 | <b>135.10</b> | 0.05 | 135.45 |
| 3 | 0.05    | [0.000, 0.100] | 23.17         | 0.00 | 38.50         | 0.00 | <b>47.54</b>  | 0.24 | 50.45  |
|   |         | [0.025, 0.075] | 22.84         | 0.00 | 38.36         | 0.00 | <b>48.58</b>  | 0.20 | 51.59  |
|   |         | [0.050, 0.050] | 22.65         | 0.00 | 38.35         | 0.00 | <b>48.14</b>  | 0.21 | 50.96  |
|   | 0.10    | [0.050, 0.150] | 38.94         | 0.00 | <b>149.82</b> | 0.00 | <b>149.82</b> | 0.00 | 149.82 |
|   |         | [0.075, 0.125] | 39.06         | 0.00 | <b>149.83</b> | 0.00 | <b>149.83</b> | 0.00 | 149.83 |
|   |         | [0.100, 0.100] | 38.05         | 0.00 | <b>149.81</b> | 0.00 | <b>149.81</b> | 0.00 | 149.81 |
|   | 0.05    | [0.000, 0.100] | 28.87         | 0.00 | 146.44        | 0.00 | <b>146.54</b> | 0.01 | 145.54 |
|   |         | [0.025, 0.075] | 27.86         | 0.00 | 146.41        | 0.00 | <b>146.48</b> | 0.01 | 146.48 |
|   |         | [0.050, 0.050] | 27.95         | 0.00 | 146.88        | 0.00 | <b>146.91</b> | 0.01 | 146.91 |

Table 2: In each row of the following table, we test on 100 random graphs, each of them has 150 vertices. We create two copies of our program (with  $k$ -DN), one takes the solution of DROP as the initial lower bound and the other takes the solution of IDROP as the initial lower bound. We list the results of the average running time and the average number of leaves in the search tree for finding a maximum  $k$ -club for  $k = 2, 3, 4$  on those random graphs.

| k | density | [a, b]         | Exact (DROP) |           | Exact (IDROP) |           |
|---|---------|----------------|--------------|-----------|---------------|-----------|
|   |         |                | time         | # leaves  | time          | # leaves  |
| 2 | 0.05    | [0.000, 0.100] | 0.045        | 148.01    | 0.116         | 77.29     |
|   |         | [0.025, 0.075] | 0.046        | 147.36    | 0.117         | 78.60     |
|   |         | [0.050, 0.050] | 0.044        | 146.44    | 0.116         | 79.11     |
|   | 0.10    | [0.050, 0.150] | 1.043        | 3966.11   | 1.020         | 3358.51   |
|   |         | [0.075, 0.125] | 1.017        | 3815.53   | 1.005         | 3152.54   |
|   |         | [0.100, 0.100] | 1.032        | 3778.68   | 1.008         | 3250.52   |
|   | 0.15    | [0.100, 0.200] | 353.475      | 848169.95 | 334.177       | 843763.15 |
|   |         | [0.125, 0.175] | 397.429      | 972089.61 | 389.458       | 962774.88 |
|   |         | [0.150, 0.150] | 333.674      | 904759.61 | 318.306       | 900947.68 |
|   | 0.20    | [0.100, 0.300] | 0.137        | 168.73    | 0.157         | 132.53    |
|   |         | [0.150, 0.250] | 0.156        | 190.74    | 0.139         | 117.77    |
|   |         | [0.200, 0.200] | 0.165        | 199.16    | 0.161         | 115.35    |
| 3 | 0.05    | [0.000, 0.100] | 6.823        | 11386.31  | 5.262         | 10541.12  |
|   |         | [0.025, 0.075] | 6.489        | 11047.96  | 4.406         | 10067.71  |
|   |         | [0.050, 0.050] | 6.651        | 11324.81  | 4.485         | 10212.02  |
|   | 0.10    | [0.050, 0.150] | 0.000        | 1.00      | 0.002         | 1.00      |
|   |         | [0.075, 0.125] | 0.000        | 1.00      | 0.002         | 1.00      |
|   |         | [0.100, 0.100] | 0.000        | 1.00      | 0.002         | 1.00      |
| 4 | 0.05    | [0.000, 0.100] | 0.000        | 1.36      | 0.009         | 1.03      |
|   |         | [0.025, 0.075] | 0.001        | 1.50      | 0.011         | 1.21      |
|   |         | [0.050, 0.050] | 0.000        | 1.17      | 0.007         | 1.04      |

Table 3: In each row of the following table, we test on 100 random graphs, each of them has 150 vertices. We create two copies of our program (with  $k$ -DN), one includes the data reduction rule and the other does not. We list the results of the average running time and the average number of leaves in the search tree for finding a maximum 2-club on those random graphs.

| density | [a, b]         | with data reduction |           | without data reduction |            |
|---------|----------------|---------------------|-----------|------------------------|------------|
|         |                | time                | # leaves  | time                   | # leaves   |
| 0.05    | [0.000, 0.100] | 0.116               | 77.29     | 0.102                  | 79.21      |
|         | [0.025, 0.075] | 0.117               | 78.60     | 0.104                  | 80.44      |
|         | [0.050, 0.050] | 0.116               | 77.11     | 0.105                  | 80.77      |
| 0.10    | [0.050, 0.150] | 1.020               | 3358.51   | 1.002                  | 3781.51    |
|         | [0.075, 0.125] | 1.005               | 3152.54   | 0.979                  | 3535.09    |
|         | [0.100, 0.100] | 1.008               | 3250.52   | 1.053                  | 3665.53    |
| 0.15    | [0.100, 0.200] | 334.177             | 843763.15 | 329.875                | 903219.66  |
|         | [0.125, 0.175] | 389.458             | 962774.88 | 393.678                | 1034288.80 |
|         | [0.150, 0.150] | 318.306             | 900947.68 | 365.604                | 967441.72  |
| 0.20    | [0.100, 0.300] | 0.157               | 132.53    | 0.178                  | 134.64     |
|         | [0.150, 0.250] | 0.139               | 117.77    | 0.173                  | 118.32     |
|         | [0.200, 0.200] | 0.136               | 115.35    | 0.167                  | 115.85     |

Table 4: This table presents the average running time of programs with k-DN and without k-DN spent to find a maximum k-club in 100 random graphs ( $|V| = 150$ ). Both programs do not apply the data reduction rule. For each test case, we use bold texts to mark the average running time that is less than the other.

| k | Density | [a, b]         | without k-DN | with k-DN      |
|---|---------|----------------|--------------|----------------|
|   |         |                | Time         | Time           |
| 2 | 0.050   | [0.000, 0.100] | 3.426        | <b>0.102</b>   |
|   |         | [0.025, 0.075] | 2.620        | <b>0.104</b>   |
|   |         | [0.050, 0.050] | 3.630        | <b>0.105</b>   |
|   | 0.100   | [0.050, 0.150] | 14.723       | <b>1.002</b>   |
|   |         | [0.075, 0.125] | 14.598       | <b>0.979</b>   |
|   |         | [0.100, 0.100] | 14.563       | <b>1.053</b>   |
|   | 0.150   | [0.100, 0.200] | 1966.253     | <b>329.875</b> |
|   |         | [0.125, 0.175] | 2268.970     | <b>393.678</b> |
|   |         | [0.150, 0.150] | 2022.607     | <b>365.604</b> |
|   | 0.200   | [0.100, 0.300] | 5.431        | <b>0.178</b>   |
|   |         | [0.150, 0.250] | 5.265        | <b>0.173</b>   |
|   |         | [0.200, 0.200] | 5.228        | <b>0.167</b>   |
| 3 | 0.050   | [0.000, 0.100] | 49.109       | <b>5.230</b>   |
|   |         | [0.025, 0.075] | 47.554       | <b>4.157</b>   |
|   |         | [0.050, 0.050] | 40.153       | <b>4.177</b>   |
|   | 0.100   | [0.050, 0.150] | 0.118        | <b>0.003</b>   |
|   |         | [0.075, 0.125] | 0.113        | <b>0.002</b>   |
|   |         | [0.100, 0.100] | 0.125        | <b>0.002</b>   |
| 4 | 0.050   | [0.000, 0.100] | 1.606        | <b>0.006</b>   |
|   |         | [0.025, 0.075] | 1.635        | <b>0.006</b>   |
|   |         | [0.050, 0.050] | 1.448        | <b>0.007</b>   |

Table 5: This table presents the size of the maximum k-clubs,  $k = 2, 3, 4$ , in some DIMACS graphs found by our program (with k-DN and the reduction rule) based on the branch-and-bound algorithm described in Section 3 and the solution found by IDROP as the initial lower bound. We also list the solution size of IDROP and DROP.

| Graph    | Density | n   | k | Exact |       | IDROP |       | DROP |      |
|----------|---------|-----|---|-------|-------|-------|-------|------|------|
|          |         |     |   | Size  | Time  | Size  | Time  | Size | Time |
| c-fat200 | 0.08    | 200 | 2 | 18    | 0.01  | 18    | 0.01  | 18   | 0.00 |
|          |         |     | 3 | 24    | 0.04  | 24    | 0.03  | 24   | 0.00 |
|          |         |     | 4 | 30    | 0.06  | 30    | 0.06  | 30   | 0.00 |
|          | 0.16    | 200 | 2 | 35    | 0.07  | 35    | 0.07  | 35   | 0.00 |
|          |         |     | 3 | 46    | 0.21  | 46    | 0.18  | 46   | 0.00 |
|          |         |     | 4 | 57    | 0.40  | 57    | 0.36  | 57   | 0.00 |
|          | 0.43    | 200 | 2 | 87    | 0.70  | 87    | 0.59  | 87   | 0.00 |
|          |         |     | 3 | 200   | 0.03  | 200   | 0.01  | 200  | 0.00 |
|          |         |     | 4 | 200   | 0.04  | 200   | 0.00  | 200  | 0.00 |
|          | 0.04    | 500 | 2 | 21    | 0.08  | 21    | 0.05  | 21   | 0.00 |
|          |         |     | 3 | 28    | 0.13  | 28    | 0.13  | 28   | 0.00 |
|          |         |     | 4 | 35    | 0.24  | 35    | 0.23  | 35   | 0.00 |
| c-fat500 | 0.07    | 500 | 2 | 39    | 0.35  | 39    | 0.26  | 39   | 0.00 |
|          |         |     | 3 | 52    | 0.64  | 52    | 0.62  | 52   | 0.00 |
|          |         |     | 4 | 65    | 1.31  | 65    | 1.29  | 65   | 0.00 |
|          | 0.19    | 500 | 2 | 96    | 3.59  | 96    | 2.80  | 96   | 0.00 |
|          |         |     | 3 | 128   | 7.81  | 128   | 7.73  | 128  | 0.00 |
|          |         |     | 4 | 159   | 19.25 | 159   | 18.37 | 159  | 0.00 |

Table 6: This table presents the size of the maximum k-clubs,  $k = 2, 3, 4$ , in Erdős networks found by our program (with k-DN and the reduction rule) based on the branch-and-bound algorithm described in Section 3 and the solution found by IDROP as the initial lower bound. We also list the solution size of IDROP and DROP. For those cases that the system has insufficient memory space, we use '-' to mark them.

| Graph      | Density | n    | k | Exact |        | IDROP |         | DROP |      |
|------------|---------|------|---|-------|--------|-------|---------|------|------|
|            |         |      |   | Size  | Time   | Size  | Time    | Size | Time |
| Erdos-97-1 | 0.012   | 472  | 2 | 42    | 0.08   | 42    | 0.08    | 35   | 0.00 |
|            |         |      | 3 | 117   | 0.78   | 117   | 0.57    | 117  | 0.00 |
|            |         |      | 4 | 235   | 0.70   | 235   | 0.68    | 235  | 0.00 |
| Erdos-98-1 | 0.012   | 485  | 2 | 43    | 0.09   | 43    | 0.09    | 36   | 0.00 |
|            |         |      | 3 | 123   | 0.84   | 123   | 0.62    | 121  | 0.00 |
|            |         |      | 4 | 244   | 1.45   | 244   | 1.41    | 243  | 0.00 |
| Erdos-99-1 | 0.012   | 492  | 2 | 43    | 0.10   | 43    | 0.09    | 37   | 0.00 |
|            |         |      | 3 | 126   | 0.79   | 126   | 0.61    | 125  | 0.00 |
|            |         |      | 4 | 245   | 1.89   | 245   | 1.53    | 245  | 0.00 |
| Erdos-97-2 | 0.001   | 5488 | 2 | 258   | 1.50   | 258   | 0.43    | 258  | 0.00 |
|            |         |      | 3 | 517   | 246.65 | 517   | 192.06  | 517  | 0.00 |
|            |         |      | 4 | -     | -      | 1504  | 1498.12 | 1504 | 0.00 |
| Erdos-98-2 | 0.001   | 5822 | 2 | 274   | 1.69   | 274   | 0.50    | 274  | 0.00 |
|            |         |      | 3 | 547   | 311.97 | 547   | 309.27  | 547  | 0.00 |
|            |         |      | 4 | -     | -      | 1594  | 1863.62 | 1581 | 0.00 |
| Erdos-99-2 | 0.001   | 6100 | 2 | 277   | 1.86   | 277   | 0.55    | 277  | 0.00 |
|            |         |      | 3 | 562   | 315.79 | 562   | 255.14  | 562  | 0.00 |
|            |         |      | 4 | -     | -      | 1643  | 2199.42 | 1631 | 0.00 |